Django Advanced Notes: Database Transactions, Custom Managers & ORM Optimization

Prepared by: Faruk Ahmed

Topics: Django ORM, Database Transactions, Custom Managers, Query Optimization

Level: Beginner to Advanced

Language: English

Table of Contents

1. Database Transactions
2. ACID Properties
3. Transactions in Django
4. Nested Transactions and Savepoints
5. Autocommit and `ATOMIC_REQUESTS`
6. Custom Model Managers
7. Custom QuerySets and `as_manager()`
8. Manager vs. QuerySet
9. ORM Optimization
10. The N+1 Query Problem
11. `select_related()` and `prefetch_related()`
12. Advanced `Prefetch()`
13. Aggregation and Annotation
14. Field Selection
15. `F()` Expressions
16. `Subquery()`, `OuterRef()`, and `Exists()`
17. Bulk Operations
18. Database Indexing
19. Row Locking with `select_for_update()`
20. QuerySet Lazy Evaluation
21. Pagination and Memory Optimization
22. Query Debugging and Execution Plans
23. Real E-commerce Optimization
24. Common Mistakes

25. Interview Questions and Answers
26. Final Revision Checklist

Part 1 — Database Transactions

1. What Is a Database Transaction?

A database transaction is a logical unit of one or more database operations that should succeed together or be rolled back when a failure occurs.

Common database operations include:

- `SELECT`
- `INSERT`
- `UPDATE`
- `DELETE`

A transaction can contain both read and write operations, although read-only queries do not normally modify database data.

Example: Bank Transfer

Suppose we transfer 500 units of currency from Account A to Account B.

1. Deduct 500 from Account A.
2. Add 500 to Account B.

If both operations succeed, commit the transaction.

If an error prevents the transfer from completing, roll back the transaction.

Without a transaction, Account A could lose money without Account B receiving it.

2. SQL Transaction Example

```
BEGIN;

UPDATE accounts
SET balance = balance - 500
WHERE id = 1;

UPDATE accounts
SET balance = balance + 500
WHERE id = 2;

COMMIT;
```

`COMMIT` makes the transaction's changes permanent according to the database's transaction guarantees.

If the transaction needs to be aborted before commit:

```
ROLLBACK;
```

Important: A transaction does not automatically validate every business rule. Application logic, database constraints, and appropriate error handling are still necessary.

Part 2 — ACID Properties

ACID describes four important properties of database transactions.

1. Atomicity

Atomicity means that the operations in a transaction are treated as one logical unit.

- All required operations succeed and the transaction commits.
- If the transaction is rolled back, its uncommitted changes are undone.

Example:

```
Account A = 2000
Account B = 1000
Transfer   = 500

After success:
Account A = 1500
Account B = 1500
```

If the transaction fails and rolls back:

```
Account A = 2000
Account B = 1000
```

2. Consistency

Consistency means that a successful transaction preserves the database's defined constraints and business rules.

Examples:

- Account balances must satisfy applicable rules.
- Foreign keys must reference valid records.
- Unique fields must not contain duplicate values.

Example: Prevent Negative Balances

```
from django.db import models
```

```
class Account(models.Model):
    balance = models.DecimalField(
        max_digits=12,
        decimal_places=2,
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(balance__gte=0),
                name="account_balance_nonnegative",
            ),
        ]
```

This constraint helps prevent negative balances at the database level on supported backends.

3. Isolation

Isolation controls how concurrent transactions interact.

Example:

Two customers try to purchase the last available product at the same time. Without appropriate concurrency control, both operations might incorrectly proceed.

Possible techniques include:

- Conditional database updates
- Row locking
- Transactions
- Appropriate transaction isolation levels
- Database constraints

The actual behavior depends on the database engine and its configuration.

4. Durability

Durability means that successfully committed changes persist according to the database's durability guarantees.

Example:

```
Payment verified
      ↓
Transaction committed
```

```
      ↓
Order marked as paid
```

Once a transaction commits, an ordinary application error does not automatically undo it.

ACID Revision

```
A — Atomicity: All or nothing
C — Consistency: Preserve defined rules
I — Isolation: Control concurrent interactions
D — Durability: Committed changes persist
```

Part 3 — Database Transactions in Django

1. `transaction.atomic()`

Django provides `transaction.atomic()` to group database operations into a transaction.

```
from django.db import transaction


with transaction.atomic():
    # Perform related database operations.
    pass
```

When the block completes successfully, Django commits the outermost transaction, or releases the relevant savepoint if the block is nested.
If an exception escapes the block, Django rolls back the relevant transaction or savepoint.

2. Bank Transfer Example

Assume an `Account` model contains a `balance` field.

```
from django.db import transaction
from django.db.models import F

from .models import Account


@transaction.atomic
def transfer_money(from_id, to_id, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive.")
```

```
    if from_id == to_id:
        raise ValueError("Accounts must be different.")

    updated = Account.objects.filter(
        pk=from_id,
        balance__gte=amount,
    ).update(
        balance=F("balance") - amount
    )

    if updated != 1:
        raise ValueError(
            "Sender account not found or insufficient balance."
        )

    recipient_updated = Account.objects.filter(
        pk=to_id,
    ).update(
        balance=F("balance") + amount
    )

    if recipient_updated != 1:
        raise Account.DoesNotExist(
            "Recipient account not found."
        )
```

How It Works

1. Validates that the transfer amount is positive.
2. Rejects transfers between the same account.
3. Deducts money only if the sender has sufficient balance.
4. Adds money to the recipient's account.
5. Commits both changes if the function completes successfully.
6. Rolls back the deduction if an exception escapes the transaction.

`F()` expressions perform arithmetic at the database level. The conditional update also avoids relying on a stale balance value read earlier in Python.

Production warning: A real financial application needs additional safeguards, including suitable concurrency control, ledger records, idempotency, account-status validation, currency precision, and carefully designed locking.

3. Proper Exception Handling

A useful pattern is to catch database exceptions outside the relevant `atomic()` block.

```
from django.db import DatabaseError, transaction


try:
    with transaction.atomic():
        perform_database_updates()
except DatabaseError:
    # Log the error and handle the failure appropriately.
    pass
```

`perform_database_updates()` is a placeholder for application code.

Avoid swallowing errors silently in production. Log relevant information and handle expected exceptions specifically.

Important: After a database error breaks a transaction, do not continue issuing queries inside that broken transaction. Exit the relevant `atomic()` block so that rollback can occur first.

Part 4 — Nested Transactions and Savepoints

Django supports nested `atomic()` blocks.

```
from django.db import transaction


with transaction.atomic():
    # Outer transaction.

    with transaction.atomic():
        # Inner transaction / savepoint.
        pass
```

A nested `atomic()` block normally creates a savepoint when savepoints are supported and enabled.

A savepoint allows Django to roll back part of a transaction without necessarily rolling back the entire outer transaction.

Example

```
from django.db import transaction


with transaction.atomic():
    create_order()

    try:
        with transaction.atomic():
            create_optional_coupon()
    except ValueError:
        print("Coupon operation failed.")

    create_order_history()
```

Here, if `create_optional_coupon()` raises a `ValueError`, the inner block rolls back and the exception is caught outside that inner block. The outer transaction can continue. If another uncaught exception escapes the outer block, the entire outer transaction is rolled back.

Remember:

- Rolling back to a savepoint is not the same as rolling back the entire transaction.
- A successful inner block does not independently commit the outer transaction.

Part 5 — Autocommit and `ATOMIC_REQUESTS`

1. Django Autocommit

Django normally uses autocommit mode.

```
Account.objects.filter(
    pk=1
).update(
    balance=1000
)
```

Outside an explicit transaction, a successful database write is normally committed automatically.

To group related operations:

```
with transaction.atomic():
    operation_one()
```

```
    operation_two()
    operation_three()
```

2. `ATOMIC_REQUESTS`

In Django settings, `ATOMIC_REQUESTS=True` for a database can wrap each view's execution in a transaction.

Conceptual configuration:

```
DATABASES = {
    "default": {
        # Database configuration...
        "ATOMIC_REQUESTS": True,
    },
}
```

Important Considerations

- Wrapping every view in a transaction introduces overhead.
- Middleware processing and template rendering are not necessarily inside the view transaction.
- Streaming responses can perform work after the view transaction has completed.
- Transactions should be chosen according to application requirements.

Part 6 — Custom Model Managers

1. What Is a Model Manager?

A Django Model Manager provides the interface used to initiate database queries for a model.

The default manager is normally named `objects`.

```
Product.objects.all()

Product.objects.filter(
    is_active=True
)

Product.objects.get(pk=1)
```

2. What Is a Custom Manager?

A custom manager is a class derived from `models.Manager` that provides reusable model-querying logic.

Benefits:

- Reduces duplicated query code.
- Makes queries easier to read.
- Organizes reusable filters.
- Provides convenient access to common subsets of data.

3. Creating a Custom Manager

```
from django.db import models


class ActiveProductManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(
            is_active=True
        )


class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    is_active = models.BooleanField(default=True)

    objects = models.Manager()
    active = ActiveProductManager()

    def __str__(self):
        return self.name
```

Usage:

```
all_products = Product.objects.all()

active_products = Product.active.all()
```

Suppose the database contains:

ID	Name	Active
1	Laptop	True
2	Mouse	False
3	Keyboard	True

Then:

```
Product.objects.all()
```

Returns all three products.

```
Product.active.all()
```

Returns Laptop and Keyboard.

Warning: Filtering the default manager can cause records to be unexpectedly excluded from some operations. Keep the default manager behavior in mind when designing related-object access and framework integrations.

4. Custom Manager Methods

You can add methods without overriding `get_queryset()`.

```
class ProductManager(models.Manager):
    def active_products(self):
        return self.get_queryset().filter(
            is_active=True
        )

    def inactive_products(self):
        return self.get_queryset().filter(
            is_active=False
        )

    def expensive_products(self, minimum_price=1000):
        return self.get_queryset().filter(
            price__gt=minimum_price
        )
```

Attach the manager:

```
class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    is_active = models.BooleanField(default=True)

    objects = ProductManager()
```

Usage:

```
active = Product.objects.active_products()

inactive = Product.objects.inactive_products()

expensive = Product.objects.expensive_products(
    5000
)
```

Part 7 — Custom QuerySets and `as_manager()`

1. What Is a Custom QuerySet?

A custom QuerySet lets you create reusable, chainable query methods.

```
from django.db import models


class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)

    def inactive(self):
        return self.filter(is_active=False)

    def expensive(self, minimum_price=1000):
        return self.filter(
            price__gt=minimum_price
        )
```

Attach it to a model:

```
class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    is_active = models.BooleanField(default=True)

    objects = ProductQuerySet.as_manager()
```

Usage:

```
Product.objects.active()

Product.objects.inactive()

Product.objects.active().order_by("name")

Product.objects.active().expensive(5000)
```

The final example returns active products priced above 5000.

2. Why Use `as_manager()`?

```
objects = ProductQuerySet.as_manager()
```

This exposes suitable public QuerySet methods through a Manager, allowing you to write reusable query logic without duplicating every method in a separate manager class.

3. Manager vs. QuerySet

Feature	Manager	QuerySet
Main purpose	Start queries	Build and filter querysets
Typical access	`Product.objects`	`Product.objects.all()`

Custom methods	Supported	Supported
Chaining	Depends on the method	Naturally supports query chaining
Created by	`models.Manager`	`models.QuerySet`

Golden Rule

- Use a Manager to provide a convenient query entry point.
- Use a QuerySet for reusable, chainable filtering logic.
- Use a service layer for complex multi-step business workflows when appropriate.

Part 8 — The N+1 Query Problem

1. What Is ORM Optimization?

Django ORM optimization improves database performance by reducing unnecessary queries, excessive data retrieval, expensive database work, and unnecessary Python processing.

Optimization should consider:

- Query count
- Query execution time
- Data transferred from the database
- Memory consumption
- Database workload
- Response latency
- Correctness under concurrency

2. Example Models

```
from django.db import models
```

```
class Author(models.Model):
    name = models.CharField(max_length=100)


class Book(models.Model):
    title = models.CharField(max_length=200)

    author = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name="books",
    )
```

3. Inefficient Query Pattern

```
books = Book.objects.all()

for book in books:
    print(book.title, book.author.name)
```

If author relationships are not already loaded, a typical query pattern is:

```
1 query  → Books
N queries → Authors

Potential total: N + 1 queries
```

If 100 books are retrieved, the code may generate 101 queries.

If several books share an author, relationship caching may reduce the actual number of queries, but the N+1 problem can still occur.

Why Is It a Problem?

- More database round trips
- Slower pages and APIs
- Increased database workload
- Greater use of database connections and server resources

The primary solutions are `select_related()` and `prefetch_related()`.

Part 9 — `select_related()`

`select_related()` retrieves related objects using SQL joins. It is primarily designed for single-valued relationships.

Typical relationships:

- `ForeignKey`
- `OneToOneField`

Example

```
books = Book.objects.select_related("author")

for book in books:
    print(book.title, book.author.name)
```

Django can retrieve books and their authors in one SQL query.

Conceptual SQL:

```
SELECT
    book.id,
    book.title,
    book.author_id,
    author.id,
    author.name
FROM book
INNER JOIN author
    ON book.author_id = author.id;
```

The actual SQL depends on the model configuration and database backend.

Multiple Relationships

```
payments = Payment.objects.select_related(
    "order__user"
)
```

This can load the payment, its order, and the order's user through joins, assuming those relationship fields exist.

When Should You Use It?

Use `select_related()` when you need:

- A book's author
- A product's category
- An order's customer
- A payment's order and user
- Another suitable single-valued relationship

Avoid adding unnecessary joins to long relationship chains. Extra joins can increase query complexity and the amount of data transferred.

Part 10 — `prefetch_related()`

`prefetch_related()` is primarily used for relationships that can contain multiple related objects.

Common use cases:

- `ManyToManyField`
- Reverse `ForeignKey`
- Other suitable multi-valued relationships

Django generally executes separate queries and associates the results in Python.

Example: Product Images

```
class Product(models.Model):
    name = models.CharField(max_length=100)


class ProductImage(models.Model):
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
        related_name="images",
    )

    image = models.ImageField(
        upload_to="products/"
    )
```

Without optimization:

```
products = Product.objects.all()

for product in products:
    for image in product.images.all():
        print(image.image.url)
```

A typical query pattern for 100 products is:

```
1 query   → Products
100 queries → Images
```

```
Potential total: 101 queries
```

With optimization:

```
products = Product.objects.prefetch_related(
    "images"
)

for product in products:
    for image in product.images.all():
        print(image.image.url)
```

A typical query pattern becomes:

```
1 query → Products
1 query → Images

Potential total: 2 queries
```

Actual query counts vary with batching, relationships, and backend behavior.

Many-to-Many Example

```
books = Book.objects.prefetch_related("authors")

for book in books:
    print(book.title)

    for author in book.authors.all():
        print(author.name)
```

Django generally retrieves the books and authors separately, then associates them in Python.

Multiple Prefetch Operations

```
products = Product.objects.prefetch_related(
    "images",
    "reviews",
)
```

This assumes both `images` and `reviews` are valid related names.

Part 11 — Advanced `Prefetch()`

`Prefetch()` provides greater control over related querysets.

It is useful when you need to:

- Filter related objects
- Order related objects
- Store prefetched results in a separate attribute
- Optimize nested related queries

1. Filtered Prefetch

Suppose a product has many reviews, but only reviews rated 4 or higher are needed.

```
from django.db.models import Prefetch


products = Product.objects.prefetch_related(
    Prefetch(
        "reviews",
        queryset=Review.objects.filter(
            rating__gte=4
        ),
    )
)
```

This prefetches only reviews rated 4 or higher into the related-object cache.

However, a later query such as:

```
product.reviews.filter(rating=5)
```

may execute another query because it uses a different filter.

2. Using `to_attr`

When filtered data should be clearly separated from the normal related manager, use `to_attr`.

```
products = Product.objects.prefetch_related(
    Prefetch(
        "reviews",
        queryset=Review.objects.filter(
            rating__gte=4
        ),
        to_attr="good_reviews",
    )
)
```

```
for product in products:
    for review in product.good_reviews:
        print(review.rating)
```

Here, `good_reviews` is a Python list.

Benefits:

- Clearly separates filtered results.
- Avoids confusion with the normal related manager.
- Makes iteration straightforward.

3. Nested Prefetch

Suppose each order has order items and each item references a product.

```
class OrderItem(models.Model):
    order = models.ForeignKey(
        "Order",
        on_delete=models.CASCADE,
        related_name="items",
    )

    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
    )
```

A basic query:

```
orders = Order.objects.prefetch_related(
    "items__product"
)
```

An alternative combines prefetching with `select_related()`:

```
orders = Order.objects.prefetch_related(
    Prefetch(
        "items",
        queryset=OrderItem.objects.select_related(
            "product"
        ),
    )
)
```

This generally retrieves orders separately from order items joined with their products.

The most efficient option depends on the query pattern and data volume.

4. `select_related()` vs. `prefetch_related()`

Feature	`select_related()`	`prefetch_related()`
Main use	ForeignKey and OneToOne	ManyToMany and reverse ForeignKey
Technique	SQL joins	Separate queries and Python association
Query count	Often one main query	Usually multiple queries
Best for	Single related objects	Related collections
Custom filtered relations	Not its main purpose	Supported through `Prefetch()`

Simple Rule

```
ForeignKey / OneToOne
        ↓
select_related()

ManyToMany / Reverse ForeignKey
        ↓
prefetch_related()

Filtered related objects
        ↓
Prefetch()
```

Part 12 — Aggregation and Annotation

1. `annotate()`

`annotate()` adds calculated values to each object in a QuerySet.

Example: Count Reviews

```
from django.db.models import Count


products = Product.objects.annotate(
    review_count=Count("reviews")
)

for product in products:
    print(product.name, product.review_count)
```

Here, `reviews` must be a valid related name.

Django can perform the count at the database level, avoiding a separate count query for each product.

2. `aggregate()`

`aggregate()` calculates summary values for the entire QuerySet.

```
from django.db.models import Avg, Count


summary = Product.objects.aggregate(
    average_price=Avg("price"),
    total_products=Count("id"),
)
```

```
print(summary)
```

Conceptual result:

```
{
    "average_price": 2500,
    "total_products": 20,
}
```

Actual results depend on the data and field types.

3. `annotate()` vs. `aggregate()`

```
annotate()  → Calculated value for each object.
aggregate() → Summary values for the whole QuerySet.
```

4. Aggregation Warning

Joining multiple one-to-many relationships can multiply result rows and produce incorrect counts or sums.

For example:

```
Product.objects.annotate(
    review_count=Count(
        "reviews",
        distinct=True,
    )
)
```

`distinct=True` may prevent duplicate review counts caused by joins, but it is not a universal fix for every aggregation problem.

Inspect the generated SQL and verify the results.

Part 13 — Field Selection

1. `only()`

Loads specified fields initially and defers other fields until they are accessed.

```
products = Product.objects.only(
    "id",
    "name",
)
```

Accessing a deferred field later may execute another query.

```
product = Product.objects.only(
    "id",
    "name",
).get(pk=1)

print(product.name)
```

The `name` field is already loaded.

If `description` was deferred:

```
print(product.description)
```

Django may issue another query to retrieve it.

2. `defer()`

Excludes specified fields from the initial query.

```
products = Product.objects.defer(
    "description"
)
```

This can help when large text fields are not needed.

3. `values()`

Returns dictionaries containing selected field values.

```
products = Product.objects.values(
    "id",
    "name",
    "price",
)
```

Conceptual result:

```
[
    {
        "id": 1,
        "name": "Laptop",
        "price": 50000,
    },
    {
        "id": 2,
        "name": "Phone",
```

```
        "price": 20000,
    },
]
```

4. `values_list()`

Returns tuples by default.

```
products = Product.objects.values_list(
    "name",
    "price",
)
```

5. `values_list(flat=True)`

Use `flat=True` when selecting exactly one field.

```
names = Product.objects.values_list(
    "name",
    flat=True,
)
```

Conceptual result:

```
["Laptop", "Phone", "Keyboard"]
```

Comparison

Method	Output
`values()`	Dictionaries
`values_list()`	Tuples
`values_list(flat=True)`	A sequence of single-field values
`only()`	Model instances

	with deferred fields
`defer()`	Model instances with specified fields deferred

If only identifiers are needed:

```
product_ids = Product.objects.values_list(
    "id",
    flat=True,
)
```

This can avoid constructing full model instances.

Part 14 — `F()` Expressions

An `F()` expression references a database field in a query, allowing calculations without first loading the current field value into Python.

1. Decrease Stock

```
from django.db.models import F


Product.objects.filter(
    pk=1
).update(
    stock=F("stock") - 1
)
```

2. Conditional Stock Update

```
updated = Product.objects.filter(
    pk=1,
    stock__gt=0,
```

```
).update(
    stock=F("stock") - 1
)

if updated == 0:
    print("Product not found or stock unavailable.")
```

The condition and stock update occur as part of the same database update operation.

Benefits

- Avoids loading the old field value into Python.
- Performs arithmetic at the database level.
- Reduces common read-modify-write race conditions.
- Works well with bulk updates.

Important: `F()` expressions do not solve every concurrency problem. More complex workflows may require transactions, locking, and database constraints.

Part 15 — `Subquery()` and `OuterRef()`

A `Subquery()` embeds another query inside a larger query.

`OuterRef()` references a field from the outer query.

Example: Latest Review Rating

Assume each review has a `product`, `rating`, and `created_at` field.

```
from django.db.models import OuterRef, Subquery


latest_review = (
    Review.objects
    .filter(product=OuterRef("pk"))
    .order_by("-created_at", "-pk")
)

products = Product.objects.annotate(
    latest_rating=Subquery(
        latest_review.values("rating")[:1]
    )
)
```

Usage:

```
for product in products:
    print(
        product.name,
        product.latest_rating,
    )
```

Each product receives the rating of its latest review. If a product has no reviews, `latest_rating` is normally `None`.

Ordering by `-created_at` and then `-pk` provides a deterministic tiebreaker when timestamps are equal.

Part 16 — `Exists()`

`Exists()` checks whether a subquery contains at least one matching record.

Example: Check Whether a Product Has Reviews

```
from django.db.models import Exists, OuterRef


reviews = Review.objects.filter(
    product=OuterRef("pk")
)

products = Product.objects.annotate(
    has_review=Exists(reviews)
)

for product in products:
    print(
        product.name,
        product.has_review,
    )
```

Conceptual output:

```
Laptop True
Phone True
Keyboard False
```

The actual output depends on the database contents.

Retrieve Only Products with Reviews

```
products_with_reviews = Product.objects.filter(
    Exists(reviews)
)
```

Use an annotation when you need a Boolean value for every product. Use filtering when you only need products that have reviews.

Part 17 — Bulk Operations

1. `bulk_create()`

`bulk_create()` inserts multiple objects efficiently.

Inefficient Pattern

```
for i in range(1000):
    Product.objects.create(
        name=f"Product {i}",
        price=100,
    )
```

This generally executes many separate insert operations.

Optimized Pattern

```
products = [
    Product(
        name=f"Product {i}",
        price=100,
    )
    for i in range(1000)
]

Product.objects.bulk_create(
    products,
    batch_size=500,
)
```

With `batch_size=500`, Django divides the inserts into batches as necessary. It is not guaranteed to execute everything in a single SQL query.

Important Limitations

Bulk creation generally:

- Does not call each instance's `save()` method.

- Does not trigger normal per-instance save signals.
- Has backend-specific behavior regarding returned primary keys and conflict handling.
- Requires consideration of memory usage for very large lists.

2. `bulk_update()`

`bulk_update()` updates selected fields on multiple existing objects.

```
products = list(
    Product.objects.filter(
        is_active=True
    )
)

for product in products:
    product.price += 10

Product.objects.bulk_update(
    products,
    ["price"],
    batch_size=500,
)
```

This example modifies active products in Python, then updates their prices in bulk.

Limitations

- Loading objects may require an initial query.
- The update may require multiple SQL batches.
- Each object's `save()` method is not called.
- Normal per-instance save signals are not triggered.
- Memory use can be significant for large collections.

3. Prefer `update()` for Simple Database-Side Changes

If the goal is simply to increase every active product's price:

```
Product.objects.filter(
    is_active=True
).update(
    price=F("price") + 10
)
```

This performs the arithmetic at the database level and avoids loading every object into Python.

Bulk Operation Summary

Requirement	Recommended API
Insert many objects	`bulk_create()`
Update selected fields on existing objects	`bulk_update()`
Update matching rows directly	`QuerySet.update()`
Delete matching rows	`QuerySet.delete()`

Part 18 — Database Indexing

1. What Is a Database Index?

A database index helps the database locate matching records efficiently.

Indexes can improve suitable queries involving:

- `WHERE`
- `JOIN`
- `ORDER BY`

- Unique lookups

However, indexes consume storage and require maintenance during inserts, updates, and deletes.

2. Single-Field Index

```
class Product(models.Model):
    name = models.CharField(
        max_length=100,
        db_index=True,
    )
```

Use `db_index=True` when the field's query patterns justify an index.
If email addresses must be unique:

```
email = models.EmailField(
    unique=True
)
```

A uniqueness constraint normally creates an appropriate database index, depending on the backend.

3. Composite Index

```
class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    class Meta:
        indexes = [
            models.Index(
                fields=["name", "price"],
                name="product_name_price_idx",
            ),
        ]
```

This creates a composite index over `name` and `price`.
It may help queries such as:

```
Product.objects.filter(
    name="Laptop",
    price__gt=1000,
)
```

The order of fields matters. An index starting with `name` may also help queries filtering only on `name`, but it may not be suitable for queries filtering only on `price`.

4. Indexing Best Practices

Create indexes based on:

- Frequent filtering
- Common joins
- Important ordering patterns
- Uniqueness requirements
- Actual query plans

Avoid creating an index on every field.

Golden rule: Measure query performance and inspect the execution plan before adding indexes.

Part 19 — Row Locking with `select_for_update()`

On supported databases, `select_for_update()` requests locks on selected rows until the relevant transaction ends.

It is useful when concurrent operations need to coordinate changes to the same records.

Example: Reserve Product Stock

```
from django.db import transaction


def reserve_stock(product_id, quantity):
    if quantity <= 0:
        raise ValueError(
            "Quantity must be positive."
        )

    with transaction.atomic():
        product = (
            Product.objects
            .select_for_update()
```

```
            .get(pk=product_id)
        )

        if product.stock < quantity:
            raise ValueError(
                "Insufficient stock."
            )

        product.stock -= quantity

        product.save(
            update_fields=["stock"]
        )
```

Important Considerations

- Use row locking inside a transaction.
- Database backend support varies.
- SQLite does not provide the same row-level locking behavior as PostgreSQL.
- Avoid holding locks while performing slow external operations.
- Consider deadlocks and lock contention.

For simple stock deductions, a conditional update can be a useful alternative:

```
updated = Product.objects.filter(
    pk=product_id,
    stock__gte=quantity,
).update(
    stock=F("stock") - quantity
)

if updated == 0:
    raise ValueError(
        "Product not found or insufficient stock."
    )
```

This approach atomically checks and decreases stock in one update statement, but more complex business workflows may still require a transaction.

Part 20 — QuerySet Lazy Evaluation

Django QuerySets are generally lazy.

Creating a QuerySet normally does not immediately execute the database query.

```
products = Product.objects.filter(
    is_active=True
)
```

The query generally executes when the QuerySet is evaluated.

Common Evaluation Triggers

```
for product in products:
    print(product.name)
```

```
list(products)
```

```
len(products)
```

```
bool(products)
```

Other triggers include accessing individual results and converting the QuerySet to a list.

`exists()` vs. `count()`

To check whether at least one record exists:

```
if Product.objects.filter(
    is_active=True
).exists():
    print("Active products found.")
```

To count records:

```
count = Product.objects.filter(
    is_active=True
).count()
```

When you only need to check existence, `exists()` is usually more appropriate than `count() > 0`.

A QuerySet that has already been fully evaluated and cached may affect whether another database query is needed, so behavior depends on the QuerySet's state.

Part 21 — Pagination and Memory Optimization

1. Pagination

Pagination retrieves data in smaller portions instead of displaying an entire dataset at once.

```
from django.core.paginator import Paginator


products = Product.objects.order_by("id")

paginator = Paginator(products, 20)

page = paginator.get_page(1)

for product in page:
    print(product.name)
```

This displays up to 20 products on the selected page.

Important Considerations

- Use stable ordering when paginating.
- Offset-based pagination can become inefficient for very large page numbers.
- Cursor-based pagination may be appropriate for large datasets or frequently changing data.

2. `iterator()`

For large QuerySets, `iterator()` can reduce memory use by bypassing the normal QuerySet result cache.

```
for product in Product.objects.iterator(
    chunk_size=1000
):
    process_product(product)
```

`process_product()` is a placeholder.

Actual memory and query behavior depends on the database driver, backend, and related-object access patterns.

If you need related objects, consider whether prefetching or another batching strategy is appropriate.

Part 22 — Query Debugging and Execution Plans

1. `connection.queries`

During development, query logging can help inspect executed queries.

```
from django.db import connection
```

```
print(connection.queries)
```

This is useful only when the relevant query logging is enabled, commonly through `DEBUG=True`.

Limitations

- It is not a substitute for production query monitoring.
- Logs may contain sensitive query information.
- Behavior depends on the database connection and configuration.
- Large query logs can become difficult to analyze.

2. Django Debug Toolbar

Django Debug Toolbar can help identify:

- SQL queries
- Duplicate queries
- Query timings
- N+1 query problems
- Slow view behavior

Use it in development and configure access appropriately.

3. `QuerySet.explain()`

`explain()` displays the database execution plan for a QuerySet.

```
queryset = Product.objects.filter(
    is_active=True
)

print(queryset.explain())
```

Depending on the database backend, the plan may reveal:

- Table scans
- Index usage
- Join strategies
- Estimated row counts
- Other execution details

Compare query plans before and after optimization. The output format and available options depend on the backend.

Part 23 — Real E-commerce Optimization Example

Suppose an e-commerce application has:

- `Product`
- `Category`
- `ProductImage`
- `Review`

Assumptions:

- Product has a ForeignKey to Category.
- Product images use the reverse name `images`.
- Product reviews use the reverse name `reviews`.

Optimized Query

```
from django.db.models import Count, Prefetch


products = (
    Product.objects
    .select_related("category")
    .prefetch_related(
        "images",
        Prefetch(
            "reviews",
            queryset=Review.objects.filter(
                rating__gte=4
            ),
            to_attr="good_reviews",
        ),
    )
    .annotate(
        review_count=Count(
            "reviews",
            distinct=True,
        )
    )
)
```

What Has Been Optimized?

1. `select_related("category")` loads category data through a SQL join.
2. `prefetch_related("images")` retrieves product images efficiently.
3. `Prefetch()` retrieves only reviews rated 4 or higher into `good_reviews`.

4. `annotate()` calculates the review count in the database.
5. `distinct=True` helps avoid duplicate review counts caused by certain joins.

Important: Complex aggregations should still be verified. Inspect the SQL and check the results, especially when multiple one-to-many relationships are involved.

Template Example

```
{% for product in products %}
    <h2>{{ product.name }}</h2>

    <p>
        Category: {{ product.category.name }}
    </p>

    <p>
        Review Count: {{ product.review_count }}
    </p>

    {% for image in product.images.all %}
        <img
            src="{{ image.image.url }}"
            alt="{{ product.name }}"
        >
    {% endfor %}

    <h3>Good Reviews</h3>

    {% for review in product.good_reviews %}
        <p>Rating: {{ review.rating }}</p>
    {% endfor %}
{% endfor %}
```

This template assumes the model relationships and fields match the example. Remember that prefetching every related object can increase memory use. Fetch only what the page actually needs.

Part 24 — Common ORM Mistakes

Mistake 1: Using `select_related()` for Every Relationship

`select_related()` is primarily for single-valued relationships.

Use `prefetch_related()` for many-to-many relationships and reverse foreign-key collections.

Mistake 2: Filtering Again After Prefetching

```
products = Product.objects.prefetch_related(
    "reviews"
)
```

Then:

```
product.reviews.filter(
    rating__gte=4
)
```

The additional filter generally executes another query because it differs from the prefetched `.all()` result.
Use `Prefetch()` with a filtered queryset if you only need certain related objects.

Mistake 3: Assuming Bulk Operations Call `save()`

`bulk_create()` and `bulk_update()` do not follow the normal per-instance `save()` lifecycle.

Verify your application logic if it depends on model methods or signals.

Mistake 4: Indexing Every Field

Indexes increase storage and write overhead.

Choose indexes according to real query patterns and execution plans.

Mistake 5: Performing Critical Updates Without Transactions

Use `transaction.atomic()` when dependent database operations must succeed or fail together.

Mistake 6: Catching Every Exception

Avoid:

```
try:
    perform_operation()
except:
    pass
```

This hides errors and can create data-integrity problems.

Catch expected exceptions and handle failures deliberately.

Mistake 7: Optimizing Only Query Count

A low query count does not guarantee a fast query.

Consider:

- Query execution time
- Execution plans
- Rows scanned
- Data transferred
- Memory consumption
- Lock contention
- Correctness

Part 25 — Interview Questions and Answers

Q1. What is a database transaction?

A transaction is a logical unit of database operations that are committed together or rolled back when necessary.

Q2. What are ACID properties?

- **Atomicity:** All or nothing.
- **Consistency:** Preserve database rules.
- **Isolation:** Control concurrent transaction interactions.
- **Durability:** Committed changes persist.

Q3. What is `transaction.atomic()`?

It is a Django context manager and decorator for grouping database operations into a transaction.

Q4. What is a Django Model Manager?

A Manager provides the interface for initiating and organizing model queries.

Q5. What is the difference between Manager and QuerySet?

A Manager starts queries, while a QuerySet represents a collection of query results and provides filtering and chaining operations.

Q6. What is the N+1 query problem?

It occurs when one query retrieves a collection of objects and additional queries are executed for related data for each object.

Q7. What is the difference between `select_related()` and `prefetch_related()`?

`select_related()` uses SQL joins for single-valued relationships. `prefetch_related()` generally uses separate queries and associates related collections in Python.

Q8. What is `Prefetch()`?

`Prefetch()` customizes prefetching, including filtering related objects and storing results in a separate attribute using `to_attr`.

Q9. What is the difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to individual objects. `aggregate()` returns summary values for the entire QuerySet.

Q10. What is an `F()` expression?

An `F()` expression references a database field so calculations can occur at the database level without first reading the old value into Python.

Q11. What is `Exists()`?

`Exists()` is an ORM expression that checks whether a subquery contains at least one matching row.

Q12. What is `bulk_create()`?

It is a Django API for inserting multiple model objects in bulk.

Q13. What is `bulk_update()`?

It is a Django API for updating selected fields on multiple existing objects in bulk.

Q14. What is a database index?

An index is a database structure that helps locate records efficiently, at the cost of extra storage and write overhead.

Q15. How do you debug slow Django queries?

Use:

- Django Debug Toolbar
- Query logging in development
- `QuerySet.explain()`
- Database monitoring tools

- Performance measurements before and after optimization

Q16. What is the difference between `only()` and `defer()`?

`only()` specifies the fields to load initially. `defer()` specifies the fields to postpone loading.

Q17. What is the difference between `values()` and `values_list()`?

`values()` returns dictionary-like rows. `values_list()` returns tuples, or a flat sequence when `flat=True` is used with one field.

Q18. What is `select_for_update()`?

It requests row locks for selected records on supported database backends, helping coordinate concurrent operations inside a transaction.

Q19. What is QuerySet lazy evaluation?

A QuerySet generally delays executing its database query until the result is evaluated, such as through iteration or conversion to a list.

Q20. Why use a custom QuerySet?

A custom QuerySet provides reusable, chainable query methods that keep filtering logic organized and reduce duplication.

Part 26 — Final Revision Checklist

Database Transactions

- Understand transactions and ACID.
- Use `transaction.atomic()` correctly.
- Understand rollback and nested savepoints.
- Understand autocommit and `ATOMIC_REQUESTS`.
- Handle database errors appropriately.
- Protect concurrent updates using suitable techniques.

Custom Managers and QuerySets

- Understand Django Model Managers.
- Create custom manager methods.
- Create custom QuerySet methods.
- Understand `as_manager()`.

- Choose the default manager carefully.
- Keep complex business workflows organized.

ORM Optimization

- Identify N+1 query problems.
- Choose between `select_related()` and `prefetch_related()`.
- Use `Prefetch()` for filtered related data.
- Understand `annotate()` and `aggregate()`.
- Use `F()`, `Subquery()`, `OuterRef()`, and `Exists()`.
- Understand the limitations of bulk operations.
- Use `only()`, `defer()`, `values()`, and `values_list()` appropriately.
- Use pagination and batching for large datasets.
- Use `iterator()` when it suits the workload.

Database Performance

- Create indexes based on real workloads.
- Inspect execution plans with `explain()`.
- Use Django Debug Toolbar in development.
- Measure performance before and after changes.
- Consider query count, execution time, memory, and data correctness together.

Final Advice

The most important habit in Django ORM optimization is to **avoid optimizing based on assumptions**.

Follow this process:

```
1. Identify the performance problem.
2. Inspect the SQL queries.
3. Measure query count and execution time.
4. Examine the execution plan.
5. Apply the appropriate optimization.
6. Measure performance again.
7. Verify correctness and concurrency behavior.
```

A well-optimized Django application balances database performance, memory usage, maintainability, data integrity, and correctness under concurrent requests.